Docusign Envelope ID: D3F4AE77-F808-4C87-8ABF-D5530C72D98C

TWEAG
by Modus Create

Technical Review of Midnight Glacier Drop Phase 2

Tweag - Modus Create
August, 2025

Docusign Envelope ID: D3F4AE77-F808-4C87-8ABF-D5530C72D98C

Contents

1 Executive Summary and Scope **2**

2 Introduction **3**
- 2.1 Scope of this report . . . 3
- 2.2 Adaptation of the codebase for testing purposes . . . 3
  - 2.2.1 Code changes . . . 3
  - 2.2.2 Script languages . . . 4
  - 2.2.3 Hydra . . . 4
- 2.3 On signatories requirement . . . 4
- 2.4 On the claiming phase . . . 4
- 2.5 Scripts of Midnight Glacier Drop . . . 5

3 Audit **6**
- 3.1 Methodology . . . 6
- 3.2 Findings . . . 7
  - 3.2.1 Bugs . . . 8
  - 3.2.2 Design flaws . . . 10
  - 3.2.3 Code Quality . . . 14
  - 3.2.4 Unclear specification . . . 17
  - 3.2.5 Extraneous finding . . . 18
- 3.3 Conclusion . . . 18

Docusign Envelope ID: D3F4AE77-F808-4C87-8ABF-D5530C72D98C

Chapter 1

Executive Summary and Scope

This report is presented without warranty or guaranty of any type. This report lists the most salient concerns that have so far become apparent to Tweag after a partial inspection of the engineering work. Corrections, such as the cancellation of incorrectly reported issues, may arise. Therefore Tweag advises against making any business decision or other decision based on this report.

Tweag does not recommend for or against the use of any work or supplier referenced in this report. This report focuses on the technical implementation provided by the project's contractors and subcontractors, based on their information, and is not meant to assess the concept, mathematical validity, or business validity of Midnight Token Generating Entity (TGE)'s product. This report does not assess the implementation regarding financial viability nor suitability for any purpose.

Scope and Methodology

Tweag looks exclusively at the on-chain validation code provided by Midnight Token Generating Entity (TGE). This excludes all the frontend files and any problems contained therein. Tweag manually inspected the code contained in the respective files and attempted to locate potential problems in one of these categories:

a) Unclear or wrong specifications that might allow for fringe behavior.

b) Implementation that does not abide by its specification.

c) Vulnerabilities an attacker could exploit if the code were deployed as-is, either generic or specific to the project.

d) General code quality comments and minor issues that are not exploitable.

e) Design flaws that could have negative consequences in one of those areas:

- Size and efficiency of the scripts
- Relevance of the actions allowed by the protocol
- Trust relationship between the provider and the users of the product

Where applicable, Tweag will provide a recommendation for addressing the relevant issue.

Summary of findings

During the limited period of time during which Tweag inspected the product based on the above guidelines, and in this iteration of the audit, Tweag did not find any vulnerabilities in the product that could be exploited by an attacker in a harmful way. Instead, most of the issues raised in this the report fall into the later two categories of the above section: Code quality comments and Design flaws. In particular, the design of the product has been subject to a number of findings, which are not reflective of any exploitable on-chain vulnerability, regardless of their severity.

Docusign Envelope ID: D3F4AE77-F808-4C87-8ABF-D5530C72D98C

Chapter 2

Introduction

2.1 Scope of this report

This report results from the re-audit of Midnight following remediation work and additional features. Findings reported in the initial audits have either:

- been removed from this version when remediation work was completed, rendering the finding no longer relevant.
- been retained in this version when the issue persists. This may result from partial or incorrect remediation or from a decision by Midnight Token Generating Entity (TGE) not to perform any remediation.

Additionally, new findings related to the updated content have been added. These changes are numerous, with the most significant concerning the upgradability of the NIGHT minting policy and the protocol parameters.

2.2 Adaptation of the codebase for testing purposes

2.2.1 Code changes

A test suite is delivered alongside this report, which uses **cooked-validators**[1] to run traces highlighting correct and incorrect behaviors of Midnight. The codebase was not usable with `cooked-validators` out of the box, and a few changes were required.

- The `plutus-tx` version used by `cooked-validators` was more recent than the one from the codebase, so it has been updated.
- Some `INLINEABLE` pragmas were missing from the codebase, preventing successful script compilation. They have been added.
- Some unused imports have been removed.
- Some outdated script parameters have been commented out.
- Some linting pragmas have been added to avoid HLS warnings.

These changes should be harmless and were necessary to run the contracts and perform both manual and automated testing, which was a required outcome of this audit.

[1] https://github.com/tweag/cooked-validators

Docusign Envelope ID: D3F4AE77-F808-4C87-8ABF-D5530C72D98C

2.2.2 Script languages

TThe scripts of the smart contract have been written using:

- `plutarch` for one withdrawing script
- `aiken` for one minting/spending script and one withdrawing script
- `plutus-tx` for all the other scripts

To accomodate for this complicated stack, we decided to import the scripts based on their serialised version when originally written in `Plutarch` or `Aiken`. Thus, these scripts will not appear in the test suite other than in raw bytestrings.

2.2.3 Hydra

Three of the product's scripts are intended to be executed in `Hydra` which runs on `Cardano`. The advantage of Hydra is that it supports running the same logic as Cardano main chain while allowing things such as fee-free processing. Users can be relatively certain that only transactions that *would* have been permitted on-chain are allowed, while benefiting from zero transaction fees and much higher throughput.

`Hydra` is not a standard environment and thus is not directly replicable within `cooked-validators`. However, since we rely on an emulated blockchain, transactions that are supposed to run on `Hydra` were instead executed on the simulated blockchain with protocol parameters matching those used in `Hydra`. Therefore, the environment used to execute these transactions should be extremely close to the production one.

2.3 On signatories requirement

Most of the product's critical operations are protected by a multisignature (multisig) mechanism: a group of designated wallets is authorized to co-sign critical transactions. Once a predefined number of these signatories approve a transaction, it passes the multisig gate and proceeds to further validation steps. While multisig protections are common, their presence should not be viewed as a substitute for on-chain validation. If transaction integrity were to rely solely on administrative signatures, the script could delegate all checks to the signatories — a pattern that is not recommended. Human actors can behave adversarially or make honest mistakes. This is precisely why on-chain logic — enforced by smart contracts — exists. **Therefore, our analysis assumes that any verification that can be done on-chain is actually performed on-chain**. We do not treat the presence of admin signatures as a guarantee of correctness. If an expected on-chain check is missing, we will report it as a finding. The decision to address such issues is left to **TGE**'s discretion.

2.4 On the claiming phase

The claims posted in the HydraThread script (see section 2.5) are issued one at a time in Hydra. During discussions with Midnight Token Generating Entity (TGE), we learned that these claims will be posted by specific authorized parties within the Hydra head, rather than by the users who own the claims themselves. Users will submit their claims through an off-chain interface. These claims will be stored in a database and later extracted by authorized parties for posting on Hydra.

Docusign Envelope ID: D3F4AE77-F808-4C87-8ABF-D5530C72D98C

This approach appears to address Hydra's specific consensus requirements and the time constraints of the claiming phase, allowing users to submit claims off-chain even before the script is deployed. While this design is understandable, it is not reflected on-chain. In particular, the on-chain code does not specify who is authorized to post claims. As a result, some findings in this report may assume that any peer can post claims, as our analysis is based solely on the on-chain code. Such findings may be less relevant in a process fully controlled by authorized peers, and we leave this assessment to Midnight Token Generating Entity (TGE)'s discretion.

2.5 Scripts of Midnight Glacier Drop

Midnight is a collection of scripts that enable the minting and distribution of Star tokens to authorized users.

- The HydraThread script stores and aggregates claims submitted by users. It runs in Hydra and is written in `plutus-tx`.
- The Sha512 script ensures the well-formedness of a specific type of XRP claim. It runs in Hydra and is written in `plutus-tx`.
- The MerkleTrieFFI script ensures that no claim is posted more than once. It runs in Hydra and is written in `Plutarch`.
- The Night script mints the star tokens as well as the thread tokens used to identify the current claim UTxO at the HydraThread script address. It also stores the star tokens until the claiming phase ends. It runs directly on mainnet and is written in `plutus-tx`.
- The RedemptionSupply script stores the star tokens after the claiming phase ends. It allows the total supply to be split into subtrees and enables users to retrieve their star tokens once the subtrees reach the desired depth. It runs directly on mainnet and is written in `plutus-tx`.
- The Redemption script stores star tokens that have been claimed from the RedemptionSupply script but are not yet eligible to be given to users. These tokens can be unlocked from this script after a specific duration has passed, or retrieved directly from the RedemptionSupply script if that duration has already elapsed. It runs directly on mainnet and is written in `plutus-tx`.
- The ProtocolParams script mints and stores an NFT identifying the official UTxO that holds the Night parameters. These parameters are intended to be updated and include, among other values, the hash of a dedicated script guarding the minting and burning of new star tokens. It runs directly on mainnet and is written in `Aiken`.
- The MultiSig script is the initial instance of the script guarding the minting and burning of new star tokens, as specified by the official UTxO at the ProtocolParams script address. It runs directly on mainnet and is written in `Aiken`.

Docusign Envelope ID: D3F4AE77-F808-4C87-8ABF-D5530C72D98C

Chapter 3

Audit

3.1 Methodology

Tweag analyzed the scripts comprising the Midnight protocol as of commit `e283f8..276218` of the repository[1] shared with us. The audit was conducted in two phases, with the second phase coming shortly after the first, and adding new scripts while correcting some of the first phase findings. We will publish copies of this report at the end of both phases. Our analysis is based on manual reviewing as well as programmatic testing using our open source tool, `cooked-validators`(`https://github.com/tweag/cooked-validators`). This programmatic testing required small updates to the audited codebase, which are depicted in Section 2.2.1. The names of the files considered in either phases of this audit are listed in Table 3.1.

File Name
mgdoc/src/Midnight/GlacierDrop/Api.hs
mgdoc/src/Midnight/GlacierDrop/FixShow.hs
mgdoc/src/Midnight/GlacierDrop/JSON.hs
mgdoc/src/Midnight/GlacierDrop/NightApi.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Common.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/HydraThread/Terms.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/HydraThread/Types.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/MerkleTrieFFI/Terms.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/MerkelTrieFFI/Types.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Night/Terms.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Night/Types.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Redemption/Terms.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Redemption/Types.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/RedemptionSupply/Terms.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/RedemptionSupply/Types.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Supply/Bech32.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Supply/Bitcoin/BIP0137.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Supply/Bitcoin/BIP0322.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Supply/Bitcoin/Types.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Supply/Bitcoin.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Supply/Byte.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Supply/Cardano/AddressBech32.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Supply/Cardano/CoseSign1.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Supply/Cardano.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Supply/CBOR.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Supply/Ethereum.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Supply/Hex.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Supply/Solana.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Supply/XRP/SHA512.hs
mgdoc/src/Midnight/GlacierDrop/Scripts/Supply/XRP.hs
protocol-params/validators/multisig.ak
protocol-params/validators/protocol-params.ak
protocol-params/lib/types.ak
protocol-params/lib/utils.ak

Table 3.1: *Files analyzed*

[1] `https://github.com/input-output-hk/mgdoc-claim-enablement`

Docusign Envelope ID: D3F4AE77-F808-4C87-8ABF-D5530C72D98C

Our analysis is based on the original documentation provided by **TGE**, Slack conversations and weekly meetings with **TGE**.

3.2 Findings

Table 3.2 lists our concerns with the current Midnight Token Generating Entity (TGE)'s code base based on our partial exploration during a limited period of time. Throughout the rest of this section, we will detail each of our findings.

Severity	Section	Summary
High	3.2.1.1	Target address when entering hydra is unchecked
High	3.2.2.1	Tangential connection between Claim and Redemption Trees
High	3.2.2.2	Liberal actions defy on-chain validation purpose
Medium	3.2.1.2	It is impossible to subdivide at a depth smaller than 3
Medium	3.2.1.3	Wrong guard on the genesis timestamp
Medium	3.2.2.3	Lack of time boxing of the claiming phase
Medium	3.2.2.4	Hydra's unstability and multiple Hydra thread tokens
Medium	3.2.2.5	The logic behind the thawing schedule is duplicated
Low	3.2.1.4	A deadlock can occur if redemption tree depth is too small
Low	3.2.1.5	`NightBurnGenesis` cannot be executed over too big a claiming tree
Low	3.2.1.6	Script purpose for the SHA512 script is unchecked
Low	3.2.2.6	Some of the scripts share the same `TxInfo`
Low	3.2.2.7	Some redeemer and datum fields are redundant
Low	3.2.2.8	The subdivision redeemer could be optimized
Low	3.2.3.1	Incomplete pattern matching in the REDEMPTIONSUPPLY script
Low	3.2.3.2	Reference to scripts are inconsistent
Low	3.2.3.3	Inconsistent wrappers usage
Low	3.2.3.4	Remnant "to do" in equality instance
Low	3.2.4.1	Unclear specification of `intersect_length` in `utils.ak`
Low	3.2.4.2	Unclear requirements on redundant auth keys
Lowest	3.2.2.9	Useless `inputValue` in `TrieInsertRedeemer`
Lowest	3.2.3.5	SHA512 script implementation can be factored
Lowest	3.2.3.6	Inconsistent error tracing
Lowest	3.2.3.7	Inconsistent naming in continuation functions
Lowest	3.2.3.8	Duplicated helper function processing datums
Lowest	3.2.3.9	Inconsistent error messages with signatories
Lowest	3.2.5.1	Used protocol params for Hydra is non-zero for reference script cost

TABLE 3.2: *Table of findings*

Docusign Envelope ID: D3F4AE77-F808-4C87-8ABF-D5530C72D98C

3.2.1 Bugs

3.2.1.1 Target address when entering hydra is unchecked

Severity: High

Issue The claiming phase requires committing the UTxO to HydraThread first, allowing users to place their claims within Hydra. This commitment is done through the `EnterHydra` action, where a UTxO at the HydraThread script is transferred to Hydra's address. However, this transfer is not properly guarded. The script only ensures that the first output of the transaction goes to Hydra's address, but it does not guard the contents of the UTxO. As a result, a dummy payment of a few lovelaces can be placed in the first output, while the rest of the input UTxO, including the Hydra Thread token, can be redirected to any address.

Since this action is only executable by the `tgeAuthAgents` we classify this as a bug rather than a vulnerability.

Remediation We recommend checking the equality of the datum and non-ADA tokens between the HydraThread input UTxO and the output UTxO committed to Hydra.

Trace This issue is illustrated by trace *We can redirect the utxo when entering hydra* in our test suite.

3.2.1.2 It is impossible to subdivide at a depth smaller than 3

Severity: Medium

Issue During the second phase of the audit, a fix was implemented to enforce the mask size as $2^{depth-3}$. While correct in most cases, if the shard size is less than 3, the exponent becomes smaller than 0, causing the exponentiation to fail in the RedemptionSupply script. As a result, subdivision to such sizes is currently impossible.

Remediation We suggest changing the formula to $2^{\max 0(depth-3)}$ to handle these cases. This ensures that, regardless of the subtree size, the mask always contains at least one byte.

3.2.1.3 Wrong guard on the genesis timestamp

Severity: Medium

Issue The thawing schedule relies on the `GenesisTimestamp`, which marks the initial release of Star tokens into the RedemptionSupply script. This timestamp is set by the administrators when executing the transaction. Since the previous audit—where the timestamp was entirely unguarded—the logic has been updated to ensure it falls within $[x, x+6\texttt{months})$. However, x is currently expected to be the lower bound of the validity interval, which, despite being required, can still be set arbitrarily far in the past.

Remediation We recommend guarding the `GenesisTimestamp` using the upper bound of the validity interval of the associated transaction:

- `GenesisTimestamp` should be set equal to the closed upper bound of the validity interval.
- The validity interval length should be bounded by a sensible maximum value.

For example, considering a validity interval (l, r), a maximal length m for this interval, and a time t of validation, the `GenesisTimestamp` would be equal to r and guaranteed to lie within the interval $(t, t + r - l)$, and thus within $(t, t + m)$.

Docusign Envelope ID: D3F4AE77-F808-4C87-8ABF-D5530C72D98C

Trace This issue is illustrated by trace *We can run the genesis transaction with a former time stamp* in our test suite.

3.2.1.4 ■ A deadlock can occur if redemption tree depth is too small

Severity: Low

Issue Datums of UTxOs at the RedemptionSupply script address can be in two modes: `Subdividing` or `Materializing`. The first UTxO at that address can only be created in `Subdividing` mode and represents the full redemption tree. Subsequent subdivisions produce more UTxOs at the same address with smaller portions of the tree, until a target depth, `redemptionShardDepth`, which is a hardcoded parameter of the RedemptionSupply script. When the forked tree depth matches `redemptionShardDepth` after subdivision, and only then, the mode can be set to `Materializing`.

This creates an issue: if the initial tree depth is smaller or equal to `redemptionShardDepth`, no subdivision can occur, and thus no UTxO can enter `Materializing` mode. Since users can only retrieve Star tokens from UTxOs in `Materializing` mode, this leads to a deadlock where the initial RedemptionSupply UTxO becomes unspendable and users cannot redeem their tokens.

For example, if `redemptionShardDepth` is set to 7 (to keep redemption masks small), the maximum claims storable is $2^7 = 128$. If total claims are 128 or fewer, the deadlock occurs. Although expected claims are likely much higher, this is not enforced anywhere.

Remediation We suggest the following approach:

- If the full tree depth is strictly greater than `redemptionShardDepth`, Start the UTxO in `Subdividing` mode.
- If the full tree depth equals `redemptionShardDepth`, Start the UTxO directly in `Materializing` mode.
- If the full tree depth is smaller than `redemptionShardDepth`, provide an artificially completed tree with dummy leaves to fall back to the previous case. Since the redemption tree root is provided manually by administrators, the depth check could be changed to the minimum of the claiming tree depth and `redemptionShardDepth`.

Alternatively, `redemptionShardDepth` could be made dynamic rather than static, adapting to the initial redemption tree size. For example, this parameter could be included in the RedemptionSupply script datum, initially set to the minimum of the claiming tree depth and `redemptionShardDepth` and carried forward without change during subdivisions.

3.2.1.5 ■ `NightBurnGenesis` cannot be executed over too big a claiming tree

Severity: Low

Issue If the total amount of Star tokens claimed exceeds the total supply—a static script parameter—the transaction involving `NightBurnGenesis` will be rejected, as the RedemptionSupply script will not receive enough Star tokens.

Although this is a corner case—the expectation being that more Star tokens will be minted than can be claimed—this is not enforced on-chain and is therefore theoretically possible.

We are informed that this will be enforced off-chain, since the minting will follow the construction of the allotment tree.

Docusign Envelope ID: D3F4AE77-F808-4C87-8ABF-D5530C72D98C

3.2.1.6 ■ Script purpose for the Sha512 script is unchecked

Severity: Low

Issue Script `sha512HalfProxyScript` is intended to be used as a withdrawing script, but it does not explicitly enforce this restriction. While this does not pose an immediate security risk, the lack of enforcement could lead to misuse or unexpected behaviors.

Remediation We recommend adding explicit validation within the script to ensure it is used only as a withdrawing script. This constraint would help to clearly define its purpose and prevent potential misuse within the broader system.

3.2.2 Design flaws

3.2.2.1 ■ Tangential connection between Claim and Redemption Trees

Severity: High

Issue Midnight proceeds in multiple phases. The first phase, running in Hydra, allows holders of stake on multiple different blockchains to register claims of Night tokens. This results in a claim tree, which can be committed back to the main chain. Then, a redemption tree is built based on the claim tree and allows, in the next phases, staged withdrawals of the claimed Night tokens.

However, the connection between the redemption tree and the claim tree is very limited and has only decreased in the second phase of the audit. The redemption tree is created off-chain and is not checked to have any connection to the claim tree other than through the total allocated Night tokens on-chain. That is, there is no on-chain validation of the fact that claims are represented in the redemption tree, that they have the same allocated amounts or that there are not additional claims added with no representation in the claim tree. In fact, the latter situation is expected to occur in order to insert claims from the scavenger mine phase.

We believe this largely defeats the purpose of having an on-chain claiming phase, and, by extension, the purpose of using Hydra itself. Moreover, the fact that both Hydra's use and the claiming phase rely on additional flexibilities in the main chain scripts (see 3.2.2.4 and 3.2.2.2) further reinforces this concern.

Remediation We would propose that, if Hydra is to be used, full use be made of its capabilities such that the connection between claims and redemption can be validated on-chain. For example, the redemption tree could be computed in Hydra (with specific functionality to allow the scavenger mine claims to be added) and then committed back to the main chain.

Alternatively, the claims process could be done entirely off-chain, in which case the need for remminting etc may be removed.

3.2.2.2 ■ Liberal actions defy on-chain validation purpose

Severity: High

Issue Two very unconstrained actions (redeeming cases) were integrated into the product as part of the second phase of the audit:

- `DynamicMinting` for the Night script.
- `FullUpgrade` for the ProtocolParams script.

Docusign Envelope ID: D3F4AE77-F808-4C87-8ABF-D5530C72D98C

These actions are only constrained by the set of signers and do not entail any proper on-chain validation checks. `DynamicMinting` delegates all checks to a withdrawing validator, which can be changed at will with administrator privileges. `FullUpgrade` allows the protocol parameter output to be consumed for any reason without constraint.

This is problematic because such actions defy the very purpose of on-chain validation: systematically ensuring that a specific, well-defined set of transactions — and only this set — can successfully validate in the context of a given smart contract. These actions are less constrained than all others, which means the whole logic could be reduced to them. In the context of this product, users have no guarantee that the stars they own will retain value, as any amount can later be minted or burned at will, thus devaluating the asset.

Remediation These actions were implemented to accommodate future changes that are yet to be decided. Our first recommendation would be to remove them altogether. However, given these unknown requirements, we can only suggest clearly communicating to the community the reason behind these actions and their intended use, in the hope that their trust in the product will remain.

We also advise that the inherent ability of these scripts to self-upgrade is utilised as soon as possible to replace these scripts with something more constrained.

3.2.2.3 Lack of time boxing of the claiming phase

Severity: Medium

Issue The claiming phase allows users to aggregate their claims to the HydraThread official UTxO. After a certain duration, the admins can finalize the claiming phase and unlock Star tokens to the RedemptionSupply scripts, moving to subsequent phases. However, this duration is completely unguarded, giving admins full freedom to end the phase at any time—even immediately after minting—thereby preventing users from placing claims.

Since this finalisation would be carried out in the Hydra head, it would be subject to censorship by the Hydra head participants. However, we assume that in general, the participants will be relying on the on-chain logic to ensure good behaviour, rather than manually inspecting transactions.

While it is unclear whether admins have incentive to behave this way, the contracts do not forbid it.

Remediation We suggest enforcing a minimum claiming phase duration on-chain. Admins should only be permitted to finalize the claiming phase after this stipulated duration has elapsed since the UTxO was committed to Hydra.

3.2.2.4 Hydra's unstability and multiple Hydra thread tokens

Severity: Medium

Issue Hydra is relatively recent and, while offering full consensus between users and no transaction fees, suffers from instability and a likelihood of deadlocks. Consequently, the claiming process of Midnight, delegated to Hydra, is not guaranteed to terminate. The official HydraThread UTxO containing the HydraThread token may remain locked indefinitely.

To mitigate this, **TGE** implemented the ability to mint new HydraThread tokens at will to reboot the claiming process as many times as needed. The reminting checks only verify that the correct token is minted and assigned to the HydraThread script with a proper datum containing an empty set of claims.

This approach introduces multiple issues:

Docusign Envelope ID: D3F4AE77-F808-4C87-8ABF-D5530C72D98C

- Any number of such tokens can be minted.
- Minting can occur at any time, including after Star tokens have been released.
- Only one of the existing token can ever be burned without resorting to the use of `DynamicMintingLogic` (see 3.2.2.2).
- Multiple tokens with different claiming trees can coexist.
- Users cannot validate the identity of the correct set of hydra head participants.
- User claims may become obsolete after a new token is minted.
- Admins may finalize any thread, ignoring claims in others.

Although admins have little incentive to mint many tokens and confuse users, the smart contracts technically allow multiple usable tokens simultaneously.

We understand that the intended approach is to have a centralised offchain service which will submit transactions to Hydra on users' behalf, and as such obviate concerns about identifying the correct Hydra head. However, either this service is mandatory, which would make us question the usage of Hydra at all (see also 3.2.2.1), or it is merely a convenience service, in which case the above issues may pose a concern for those who choose not to use it.

Remediation We advise strengthening the reminting guards as follows:

- If the time boxing recommended in 3.2.2.3 is implemented, it should also timebox reminting, preventing HydraThread token minting after the deadline.
- Provide an action to burn extraneous tokens after the deadline.
- ReStart the schedule whenever a new official token is minted.
- Use the Night script's datum to store the name of the current official token, updating it after reminting.
- Require any action on the HydraThread script to reference the Night UTxO, ensuring only the official token is accepted.

Trace This issue is illustrated by traces

- *We can keep using old hydra threads*
- *We can remint after performing the genesis transaction*

in our test suite.

3.2.2.5 The logic behind the thawing schedule is duplicated

Severity: Medium

Issue Once the Star tokens are released into the RedemptionSupply script, they are considered frozen and begin a thawing schedule dictated by various parameters. Stars thaw slowly, becoming increasingly retrievable by users.

Users can redeem thawed Stars in two ways:

Docusign Envelope ID: D3F4AE77-F808-4C87-8ABF-D5530C72D98C

- If the full thawing schedule has elapsed, Stars can be redeemed directly from the RedemptionSupply script to the user's wallet.
- If only part of the thawing schedule has elapsed, thawed Stars can be redeemed directly from the RedemptionSupply script, while the remaining frozen Stars are sent to the Redemption script for gradual retrieval following the same schedule.

This design poses two issues of low to moderate severity:

- It is impossible for all Stars to be transferred from a RedemptionSupply script UTxO to the Redemption script; at least some Stars must have thawed. This limitation lacks motivation and relates to the more severe issue below.
- Both the Redemption and RedemptionSupply scripts embed significant thawing logic, each computing thawed and frozen amounts alongside the thawing schedule. This duplication increases code complexity and the size of both scripts.

Remediation We suggest always moving the full amount of Stars claimed by a user from the RedemptionSupply script to the Redemption script, regardless of the thawing state.

This would delegate the entire thawing schedule handling to the Redemption script, whose sole purpose is managing thawing. Consequently, the RedemptionSupply script could be simplified by removing thawing logic, focusing solely on splitting shards and releasing Stars at the correct depth.

This change would reduce the complexity and size of the RedemptionSupply script with minimal modifications to the Redemption script, which would see only a moderate increase in size. It would also clarify the distinct purposes of the two scripts.

The only drawback of this approach is that users would no longer be able to redeem all their Stars in a single step when the full thawing schedule has elapsed; instead, redemption would always require interaction with the Redemption script.

3.2.2.6 ■ Some of the scripts share the same `TxInfo`

Severity: Low

Issue The Redemption and RedemptionSupply scripts from Midnight share a single dedicated version of `TxInfo` where only the pieces of information needed by any of them are deserialized.

Remediation An improvement would be to provide each script with a tailored version of `TxInfo` that includes only the subset of attributes it requires. This would result in as many versions of `TxInfo` as there are distinct subsets of needed attributes across the project.

While this approach requires additional development effort, it could further reduce deserialization costs and improve overall efficiency.

3.2.2.7 ■ Some redeemer and datum fields are redundant

Severity: Low

Issue In several places within the codebase, the redeemer data is not part of the script's logic but is simply transferred into a datum, with the script checking equality between the two. For example, during the `HydraThreadFinalize` action on the HydraThread script, the Merkle root hash and tree depth must be provided in the redeemer, despite this information already being present in the output datum and not guarded.

Docusign Envelope ID: D3F4AE77-F808-4C87-8ABF-D5530C72D98C

While redeemers are meant to express intent and this duplication may have some value, the benefit is often negligible compared to the drawbacks of redundant data.

Remediation We recommend removing such redundant information from redeemers to eliminate unnecessary on-chain equality checks and reduce transaction size slightly.

3.2.2.8 ■ The subdivision redeemer could be optimized

Severity: Low

Issue The redeemer for the subdivision of redemption trees involved providing a list of empty trees to accommodate incomplete trees (which are the most common case). In the current implementation, the hashes of all empty subtrees on the right side of the main tree must be provided, with the expectation that their number combines with the RedemptionSupply outputs created by the transaction to total 2^n for some n. However, it is known that all empty subtrees at a given level will necessarily have the same hash, so providing them multiple times unnecessarily increases the transaction size.

Remediation We suggest modifying the redeemer to provide only a single occurrence of the empty tree hash, and duplicating this subtree as many times as necessary during verification to ensure the decomposition is well-formed.

3.2.2.9 ■ Useless `inputValue` in `TrieInsertRedeemer`

Severity: Lowest

Issue The MerkleTrieFFI script uses a Merkle forest to verify whether a claim has already been posted. It employs an associative list solely to ensure the uniqueness of keys, with the values associated to those keys being irrelevant and always set to `""`. However, the `TrieInsertRedeemer` still allows providing a value in the `inputValue` field.

Remediation We suggest removing the `inputValue` field entirely to prevent confusion and enforce that values are irrelevant in this context.

3.2.3 Code Quality

3.2.3.1 ■ Incomplete pattern matching in the RedemptionSupply script

Severity: Low

Issue Function `redemptySupplyScript` exhibits incomplete pattern matching. GHC reports this as a warning, but such warnings can be considered errors in many cases. This incomplete pattern matching arises because script purposes other than `Spending` are not handled.

Remediation We advise adopting the pattern used in other scripts, where `Minting` and `Spending` purposes are defined separately and then combined safely using functions such as `mkScript`.

Docusign Envelope ID: D3F4AE77-F808-4C87-8ABF-D5530C72D98C

3.2.3.2 ■ Reference to scripts are inconsistent

Severity: Low

Issue Throughout the codebase, and particularly in the script parameters, references to scripts are made inconsistently. Sometimes they use script hashes directly, sometimes a wrapper around script hashes, and sometimes a `Credential` which may be assumed to be a script credential.

For example, in the type `HydraThreadParams`, we observe:

- `hydraCommitScriptHash :: ScriptHash` corresponds to the hydra script.
- `sha512HalfScriptHash :: Sha512HalfScriptHash` corresponds to the SHA512 script.
- `trieInsertCred :: Credential` corresponds to the MERKLETRIEFFI script.

Similarly, in the datum of the PROTOCOLPARAMS scripts, scripts are represented in three different ways:

- The dynamic minting logic is referenced using a credential.
- The Hydra thread script is referenced using a bytestring.
- The reserve and supply scripts are referenced using addresses.

This heterogeneity is confusing and potentially misleading, as a `Credential` could technically be instantiated with a public key credential, which is undesirable.

Remediation We suggest standardizing on one approach across the codebase: either use wrappers consistently everywhere or use script hashes everywhere to ensure clarity and prevent misuse. Where pubkeys are not allowed, this should be enforced at the type level.

3.2.3.3 ■ Inconsistent wrappers usage

Severity: Low

Issue Many elements in the data used by Midnight are wrapped in newtypes with meaningful names, such as `GenesisTimestamp` wrapping a `POSIXTime`, or `MerkleTreeDepth` wrapping an integer. While using these newtypes is good practice to clarify intended usage without runtime overhead, their use is inconsistent in the codebase.

For instance, the `genesisTimestamp` field of the `NightGenesis` redeemer is typed as `POSIXTime` directly, whereas the same field in the `RedemptionSupplyDatum` datum uses the `GenesisTimestamp` newtype. Similarly, in the `HydraThreadDatum`, the `allotmentsTreeDepth` field is typed as `Integer`, while `redemptionTreeDepth` is typed as `MerkleTreeDepth`.

This inconsistency complicates reading and writing off-chain code, leading to potential confusion and errors.

Remediation We suggest reviewing the usage of these newtype wrappers across the codebase and either removing them entirely for simplicity or applying them consistently to improve clarity and type safety.

3.2.3.4 ■ Remnant "to do" in equality instance

Severity: Low

Issue There is a remnant `traceError "to do"` in the implementation of the `Eq ScriptPurpose` instance. This causes the code to crash if it encounters script purposes that are not explicitly handled.

Docusign Envelope ID: D3F4AE77-F808-4C87-8ABF-D5530C72D98C

Remediation This instance should be updated to use catch-all patterns and return `False` when comparing script purposes that are not present in the project. This change would avoid runtime errors and optimize the equality check.

3.2.3.5 ■ Sha512 script implementation can be factored

Severity: Lowest

Issue The implementation of `sha512HalfProxyScript` is overly lengthy, which reduces code clarity and maintainability.

Remediation It can be refactored without loss of clarity to a more concise form, for example:

```
all (\(preimage, hash) -> sha512Half preimage == hash) $ unsafeFromBuiltinData hashes
```

3.2.3.6 ■ Inconsistent error tracing

Severity: Lowest

Issue The error tracing in the Plutus scripts is inconsistent, with usages of `traceIfFalse "..." False`, `traceError`, and `trace "..." error` scattered throughout the codebase. This inconsistency complicates maintenance and testing.

Remediation We advise homogenizing all error tracing calls across the scripts to use a consistent pattern. While these traces are erased in production builds, adopting a uniform approach will improve code readability, simplify debugging, and ease future maintenance.

3.2.3.7 ■ Inconsistent naming in continuation functions

Severity: Lowest

Issue The script code uses continuation style where functions perform checks while computing values and pass predicates as inputs. Most such functions are prefixed with **with**, but some, like `deserializeDatum` and `pastPointFromValidityInterval`, are not. Similarly, the `ContBool` type alias is defined but rarely used. This inconsistency hurts code readability and maintainability.

Remediation We suggest homogenizing function naming conventions by prefixing all continuation-style functions with **with**, and consistently using the `ContBool` alias where appropriate. This will improve code clarity and facilitate future maintenance.

3.2.3.8 ■ Duplicated helper function processing datums

Severity: Lowest

Issue The two functions `deserializeDatum` and `withOutputDatum` are definitionally equal. However, both coexist in the codebase, causing redundancy.

Remediation We suggest keeping only `withOutputDatum`, as its name clearly indicates it is a continuation-style function (see 3.2.3.7), and deleting `deserializeDatum`.

Docusign Envelope ID: D3F4AE77-F808-4C87-8ABF-D5530C72D98C

3.2.3.9 ■ Inconsistent error messages with signatories

Severity: Lowest

Issue Most actions before the redemption phase require multisig approval from a list of admin wallets. When these requirements are not met, two different error messages can appear: `"signatures"` and `"Not authorized"`, depending on the action. This inconsistency can cause confusion during debugging and maintenance.

Remediation We suggest unifying these errors by choosing a single error message to represent failed multisig authorization and using it consistently across all relevant actions.

3.2.4 Unclear specification

3.2.4.1 ■ Unclear specification of `intersect_length` in `utils.ak`

Severity: Low

Issue Both the MultiSig and ProtocolParams scripts require that each transaction be signed by a sufficient number of administrators. This verification relies on the `intersect_length` helper from `utils.ak`, which checks whether the intersection of the actual signers and the administrator set contains more elements than `min_auth_signatures`. However, the implementation of `intersect_length` silently assumes that both input lists are sorted. This requirement is not clear from the function's name and is not documented. Furthermore, one of the unit tests for `intersect_length` may misleadingly suggest that unsorted inputs are supported, as the left argument provided in the test is not sorted:

```
test intersect_length_3() {
  2 == intersect_length([#"aa", #"cc", #"bbbb"], [#"aa", #"bb", #"cc"])
}
```

However, this test succeeds by accident: if we swap `aa` and `cc` in the first list, it fails (returning an intersection length of `1` instead of `2`).

Given that in the worst case, `intersect_length` will underestimate the size of the intersection, this does not *per se* imply a security issue currently (transactions might be rejected if the signers are in the wrong order). Still, because `intersect_length` is used in the scripts that are critical to the security of Midnight, their specification should be crystal clear. There's also a risk of introducing vulnerabilities in future evolutions of the product.

Remediation Following the example of `sort_and_count` in the same file `utils.ak`, we advise giving a more explicit name to `intersect_length` (for example `sorted_intersect_length`), and to properly document the function and in particular its preconditions. We also advise removing or updating the misleading tests with an unsorted input.

3.2.4.2 ■ Unclear requirements on redundant auth keys

Severity: Low

Issue Most scripts in the project are parameterized with multisig requirements: a set of authorized keys and a number of required signers. It is unclear whether these sets and thresholds should be identical across all scripts or can differ. While there is now a separate protocol parameters script, this is only referenced by the NIGHT script, while the supply script uses its own parameters.

Two cases arise:

Docusign Envelope ID: D3F4AE77-F808-4C87-8ABF-D5530C72D98C

- All scripts should use the same authorized keys and signer count, which should be explicitly specified and ideally enforced during instantiation.
- Scripts can use different sets and signer counts, but this requires careful analysis of their interdependencies. Delegated checks across scripts within the same transaction could result in multisig validations against inconsistent sets, potentially causing security or usability issues.

3.2.5 Extraneous finding

3.2.5.1 ■ Used protocol params for Hydra is non-zero for reference script cost

Severity: Lowest

Issue This finding does not pertain to the on-chain code, so such issues are usually not reported. However, due to its potential negative impact on the claiming phase, we made an exception and rated it as `Lowest` severity.

In the protocol parameters provided by **TGE** for use in the emulated blockchain (simulating Hydra), all parameters related to fee computation were set to zero (as expected since Hydra transactions are intended to have zero fees), except for the cost of reference scripts. This zero-fee assumption is reflected on-chain, where the continuing output of the HydraThread script is expected to be the only output of transactions running on Hydra.

If fees were non-zero, this restriction could not be enforced because an additional output would normally be required to return the fee change, unless the entire supplied amount is set as the fee. However, if reference scripts are used with the current protocol parameters, the fee will be non-zero, and this could break the assumption.

Remediation We advise to carefully evaluate this situation and ensure that fees will indeed always be zero on Hydra, or that the protocol parameters fully and correctly reflect the fee computation. This will help avoid unexpected behavior or failures due to fee handling in Hydra transactions.

3.3 Conclusion

This report outlines the 27 concerns that we have gathered while inspecting the design and code of Midnight Token Generating Entity (TGE), pertaining to the code contained in the files listed in Table 3.1. As stated in Chapter 1, Tweag does not recommend for nor against the use of any work referenced in this report. Nevertheless, the existence of *high* and *critical* severity concerns is a warning sign.

Benjamin Robin

COO

DocuSigned by:

Benjamin Robin

67D64C89E441423...

10/10/2025